

Задание 1

Напишите небольшую основную процедуру, скомпилируйте программу `segmentation1.c`, которая у вас получилась на данный момент.

```
int main(){
    check();
    return 0;
}
```

Что выдает программа?

Что моделирует массив `memory[MEMORY]` в этом коде?

Зачем используется структура `array`? Что она описывает?

Что хранит поле `segment` — адрес или индекс? Почему это указатель?

Что означает поле `next` в структуре `array`?

Что означает выражение `nxt->segment - memory`?

Почему можно вычитать указатели?

```
vboxuser@Linux:~$ gcc Qv_10.c
```

```
vboxuser@Linux:~$ ./a.out
```

```
array (0x613c25e92030): size 1, segment -1, next 0x613c25e92010
```

```
array (0x613c25e92010): size 0, segment 100, next (nil)
```

Добавьте реализацию функции **allocate**. В полученной программе segmentation2.c выполните следующие тесты:

- Тест 1 — простое последовательное выделение

7

```
int main(){
    array *a = allocate(10);
    array *b = allocate(10);
    array *c = allocate(10);
    check();
    return 0;
}
```

При правильной реализации ожидаемый результат следующий:

```
array (0x...): size 1, segment -1, next 0x...
array (0x...): size 10, segment 0, next 0x...
array (0x...): size 10, segment 10, next 0x...
array (0x...): size 10, segment 20, next 0x...
array (0x...): size 0, segment 100, next (nil)
```

```
vboxuser@Linux:~$ gcc Qv_10.c
```

```
vboxuser@Linux:~$ ./a.out
```

```
array (0x62d680b67030): size 1, segment -1, next 0x62d68ddc92a0
array (0x62d68ddc92a0): size 10, segment 0, next 0x62d68ddc92c0
array (0x62d68ddc92c0): size 10, segment 10, next 0x62d68ddc92e0
array (0x62d68ddc92e0): size 10, segment 20, next 0x62d680b67010
array (0x62d680b67010): size 0, segment 100, next (nil)
```

- Тест 2 — отсутствие места

```
array *a = allocate(80);  
array *b = allocate(30); // должно не хватить  
printf("%p\n", b);
```

При правильной реализации ожидаемый результат: (nil)

```
vboxuser@Linux:~$ gcc Qv_10.c  
vboxuser@Linux:~$ ./a.out  
(nil)  
array (0x620debe74030): size 1, segment -1, next 0x620e097c02a0  
array (0x620e097c02a0): size 80, segment 0, next 0x620debe74010  
array (0x620debe74010): size 0, segment 100, next (nil)
```

- Тест 3 — точное совпадение по размеру

```
array *a = allocate(50);  
array *b = allocate(50);  
check();
```

```
vboxuser@Linux:~$ gcc Qv_10.c  
vboxuser@Linux:~$ ./a.out  
array (0x613757ae3030): size 1, segment -1, next 0x6137694b92a0  
array (0x6137694b92a0): size 50, segment 0, next 0x6137694b92c0  
array (0x6137694b92c0): size 50, segment 50, next 0x613757ae3010  
array (0x613757ae3010): size 0, segment 100, next (nil)
```

- Тест 4 — проверка корректности сегментов

```
array *a = allocate(10);
array *b = allocate(10);

assert(b->segment > a->segment);
assert(b->segment >= a->segment + a->size);
```

```
vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
array (0x56c2abb16030): size 1, segment -1, next 0x56c2d772d2a0
array (0x56c2d772d2a0): size 10, segment 0, next 0x56c2d772d2c0
array (0x56c2d772d2c0): size 10, segment 10, next 0x56c2abb16010
array (0x56c2abb16010): size 0, segment 100, next (nil)
```

- Тест 1 — удаление единственного блока

```
array *a = create(10);
delete(a);
check();
```

Убедитесь, что в списке только: dummy -> sentinel.

```
vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
create an array of size 10 ..done
delete array (0x62fddcd636b0) of size 10done
array (0x62fdb059030): size 1, segment -1, next 0x62fdb059010
array (0x62fdb059010): size 0, segment 100, next (nil)
```

- Тест 2 — удаление из середины

```
array *a = create(10);
array *b = create(11);
array *c = create(12);
delete(b);
check();
```

Убедитесь, что a -> c без b.

```
vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
create an array of size 10 ..done
create an array of size 11 ..done
create an array of size 12 ..done
delete array (0x5773e73dc6d0) of size 11done
array (0x5773da934030): size 1, segment -1, next 0x5773e73dc6b0
array (0x5773e73dc6b0): size 10, segment 0, next 0x5773e73dc6f0
array (0x5773e73dc6f0): size 12, segment 21, next 0x5773da934010
array (0x5773da934010): size 0, segment 100, next (nil)
```

- Тест 3 — повторное выделение после удаления

```
array *a = create(20);
array *b = create(21);
delete(a);
array *c = create(10);
check();
```

Убедитесь, что c занимает место a (segment = 0).

```
vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
create an array of size 20 ..done
create an array of size 21 ..done
delete array (0x5d913c4356b0) of size 20done
array (0x5d9139f67030): size 1, segment -1, next 0x5d913c4356d0
array (0x5d913c4356d0): size 21, segment 20, next 0x5d9139f67010
array (0x5d9139f67010): size 0, segment 100, next (nil)
```

- Тест 4 — double delete

```
array *a = create(20);  
array *b = create(21);  
delete(a);
```

```
root@kali:~/Qv_10# gcc Qv_10.c  
root@kali:~/Qv_10# ./a.out  
create an array of size 20 ..done  
create an array of size 21 ..done  
delete array (0x5d913c4356b0) of size 20done  
array (0x5d9139f67030): size 1, segment -1, next 0x5d913c4356d0  
array (0x5d913c4356d0): size 21, segment 20, next 0x5d9139f67010  
array (0x5d9139f67010): size 0, segment 100, next (nil)
```

```
void bench1(){  
    check();  
    array *a = create(20);  
    check ( ) ;  
    array *b = create(30);  
    check();  
    set(a, 10, 110);  
    set(a, 14, 114);  
    set(b, 8, 208);  
    set(b, 12, 212);  
    printf("a[10]+a[14] = %d\n", get(a, 10) + get(a, 14));  
    printf("b[8]+b[12] = %d\n", get(b, 8) + get(b, 12));  
    delete(&a);  
    check();  
    delete(&b);  
    check();  
}
```



```

vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
array (0x5ad9e0da9030): size 1, segment -1, next 0x5ad9e0da9010
array (0x5ad9e0da9010): size 0, segment 100, next (nil)
create an array of size 20 ..done
array (0x5ad9e0da9030): size 1, segment -1, next 0x5ad9e1feb6b0
array (0x5ad9e1feb6b0): size 20, segment 0, next 0x5ad9e0da9010
array (0x5ad9e0da9010): size 0, segment 100, next (nil)
create an array of size 30 ..done
array (0x5ad9e0da9030): size 1, segment -1, next 0x5ad9e1feb6b0
array (0x5ad9e1feb6b0): size 20, segment 0, next 0x5ad9e1feb6d0
array (0x5ad9e1feb6d0): size 30, segment 20, next 0x5ad9e0da9010
array (0x5ad9e0da9010): size 0, segment 100, next (nil)
a[10]+a[14] = 224
b[8]+b[12] = 420
delete array (0x5ad9e1feb6b0) of size 20done
array (0x5ad9e0da9030): size 1, segment -1, next 0x5ad9e1feb6d0
array (0x5ad9e1feb6d0): size 30, segment 20, next 0x5ad9e0da9010
array (0x5ad9e0da9010): size 0, segment 100, next (nil)
delete array (0x5ad9e1feb6d0) of size 30done
array (0x5ad9e0da9030): size 1, segment -1, next 0x5ad9e0da9010
array (0x5ad9e0da9010): size 0, segment 100, next (nil)

```

```
void bench2(){
    check();
    array *a = create(20);
    check ( ) ;
    array *b = create(30);
    check();
    set(a, 10, 110);
    set(a, 14, 114);
    set(b, 8, 208);
    set(b, 12, 212);
    printf("a[10]+a[14] = %d\n", get(a, 10) + get(a, 14));
    printf("b[8]+b[12] = %d\n", get(b, 8) + get(b, 12));
    delete(a);
    check();
    delete(b);
    check();
}
```



```

vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
array (0x5b244ffb5030): size 1, segment -1, next 0x5b244ffb5010
array (0x5b244ffb5010): size 0, segment 100, next (nil)
create an array of size 20 ..done
array (0x5b244ffb5030): size 1, segment -1, next 0x5b246fb0e6b0
array (0x5b246fb0e6b0): size 20, segment 0, next 0x5b244ffb5010
array (0x5b244ffb5010): size 0, segment 100, next (nil)
create an array of size 30 ..done
array (0x5b244ffb5030): size 1, segment -1, next 0x5b246fb0e6b0
array (0x5b246fb0e6b0): size 20, segment 0, next 0x5b246fb0e6d0
array (0x5b246fb0e6d0): size 30, segment 20, next 0x5b244ffb5010
array (0x5b244ffb5010): size 0, segment 100, next (nil)
a[10]+a[14] = 224
b[8]+b[12] = 420
delete array (0x5b246fb0e6b0) of size 20done
array (0x5b244ffb5030): size 1, segment -1, next 0x5b246fb0e6d0
array (0x5b246fb0e6d0): size 30, segment 20, next 0x5b244ffb5010
array (0x5b244ffb5010): size 0, segment 100, next (nil)
delete array (0x5b246fb0e6d0) of size 30done
array (0x5b244ffb5030): size 1, segment -1, next 0x5b244ffb5010
array (0x5b244ffb5010): size 0, segment 100, next (nil)

```

```

void bench3() {
    array *a = create(20);
    array *b = create(30);
    set(a, 22, 100);
    set(b, 0, 200);
    printf( "a[20] = %d\n", get(a, 20));
    printf( "b[0] = %d\n", get(b, 0));
    printf( "b[1] = %d\n", get(b, 1));
    printf( "b[2] = %d\n", get(b, 2));
    printf( "a[20] + b[2] = %d\n", get(a, 20)+get(b, 2));
    delete(&a);
    delete(&b);
}

```

```

vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
create an array of size  20 ..done
create an array of size  30 ..done
a[20] = 200
b[0] = 200
b[1] = 0
b[2] = 100
a[20] + b[2] = 300
delete array (0x5b256219d6b0) of size  20done
delete array (0x5b256219d6d0) of size  30done

```

```

vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
create an array of size  20 ..done
create an array of size  30 ..done
segmentation fault (core dumped)

```

Добавьте в программу следующий тест:

```
void bench4(){
    array *a = create(20);
    array *b = create(30);
    array *c = create(30);
    delete(&b);
    check();
    array *d = create(50);
    delete(&a);
    delete(&c);
    delete(&d);
}
```

```
vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
create an array of size 20 ..done
create an array of size 30 ..done
create an array of size 30 ..done
delete array (0x5c9b07b7b6d0) of size 30done
array (0x5c9ad96e7030): size 1, segment -1, next 0x5c9b07b7b6b0
array (0x5c9b07b7b6b0): size 20, segment 0, next 0x5c9b07b7b6f0
array (0x5c9b07b7b6f0): size 30, segment 50, next 0x5c9ad96e7010
array (0x5c9ad96e7010): size 0, segment 100, next (nil)
create an array of size 50 ..out of memory
```

```
vboxuser@Linux:~$ gcc Qv_10.c
vboxuser@Linux:~$ ./a.out
create an array of size 20
.. pray for the best .. ... yes!
create an array of size 30
.. pray for the best .. ... yes!
create an array of size 30
.. pray for the best .. ... yes!
delete array (0x5d7219dff6d0) of size 30done
array (0x5d71e6c93030): size 1, segment -1, next 0x5d7219dff6b0
array (0x5d7219dff6b0): size 20, segment 0, next 0x5d7219dff6f0
array (0x5d7219dff6f0): size 30, segment 50, next 0x5d71e6c93010
array (0x5d71e6c93010): size 0, segment 100, next (nil)
create an array of size 50
.. almost panic, time for compaction
array (0x5d71e6c93030): size 1, segment -1, next 0x5d7219dff6b0
array (0x5d7219dff6b0): size 20, segment 0, next 0x5d7219dff6f0
array (0x5d7219dff6f0): size 30, segment 20, next 0x5d71e6c93010
array (0x5d71e6c93010): size 0, segment 100, next (nil)
.. pray for the best .. ... yes!
delete array (0x5d7219dff6b0) of size 20done
delete array (0x5d7219dff6f0) of size 30done
delete array (0x5d7219dff6d0) of size 50done
```

Простые схемы управления памятью

Динамические разделы



40

Простые схемы управления памятью

Динамические разделы



40

Простые схемы управления памятью

Динамические разделы



41



Свободно 100, но разместить процесс №5, тем не менее, не можем.

За счет такой раздробленности происходит потеря примерно трети оперативной памяти.

41

Простые схемы управления памятью

Динамические разделы



45

Compaction

Compaction (**уплотнение памяти**) – это процесс перераспределения занятых блоков памяти для устранения фрагментации и объединения свободного пространства в один непрерывный блок

Как работает?

1. Приостановка процессов (на время перемещения).
2. Перенос всех занятых блоков в начало памяти.
3. Объединение свободного пространства в единый блок в конце.
4. Обновление указателей (адресов) в перемещённых данных

45

49

Простые схемы управления памятью

Динамические разделы



51

Простые схемы управления памятью

Динамические разделы



51

Простые схемы управления памятью

Динамические разделы

